

# LLD Notes 04 — Singleton, Functional Java, Streams, and Decorator by Composition

**Theme:** Last class we saw lambdas absorb the Strategy pattern. Today the same thing happens to Decorator — but first we need the functional toolkit that makes it possible: functional interfaces and streams. Consider this a mini Java course inside the LLD course.

---

## 1. Singleton Pattern

### The problem it solves

Some objects should exist **exactly once** per application: a logger, a configuration holder, a connection pool, a metrics registry. Creating a second one is either wasteful (two connection pools) or wrong (two loggers writing to the same file).

The recipe has two ingredients: a **private constructor** (nobody else can `new` it) and **one controlled access point**.

### Attempt 1 — Eager initialization

```
java

public class Logger {
    private static final Logger INSTANCE = new Logger();    // created at class-

    private Logger() { }                                    // nobody else can c

    public static Logger getInstance() {
        return INSTANCE;
    }
}
```

Simple, thread-safe for free (the JVM guarantees static initializers run exactly once).  
Downside: the instance is created even if it's never used — a problem only when construction is expensive.

### Attempt 2 — Lazy initialization (broken!)

"Create it only when first asked":

```
java
```

```

public class Logger {
    private static Logger INSTANCE;

    private Logger() { }

    public static Logger getInstance() {
        if (INSTANCE == null) {           // ← two threads can BOTH see null h
            INSTANCE = new Logger();      // ← and BOTH construct → two "singl
        }
        return INSTANCE;
    }
}

```

**This is not thread-safe.** Two threads call `getInstance()` at the same moment; both evaluate `INSTANCE == null` as true; both construct. Same disease as the innocent-looking `x++` — which is really *read, add, write*, three steps a thread can be interrupted between. Check-then-act is a race unless something makes it atomic.

(You can fix it by slapping `synchronized` on the method — but now every call pays for a lock, forever, to protect an initialization that happens once. There's also the famous "double-checked locking" with `volatile` — clever, subtle, easy to get wrong. Java has something better.)

### Attempt 3 — The holder idiom (lazy AND thread-safe, no locks)

```

java

public class Logger {

    private Logger() { }

    private static class Holder {           // not loaded until
        private static final Logger INSTANCE = new Logger();
    }

    public static Logger getInstance() {
        return Holder.INSTANCE;             // first call trigge
    }
}

```

The trick: the JVM loads the nested `Holder` class only when `getInstance()` first references it — and **class loading is guaranteed thread-safe by the JVM itself**. We got lazy initialization and thread safety with zero synchronization code, by delegating the hard part to the runtime.

## Attempt 4 — The enum (Joshua Bloch's recommendation)

```
java

public enum Logger {
    INSTANCE;

    public void log(String message) {
        System.out.println("[LOG] " + message);
    }
}

// usage
Logger.INSTANCE.log("started");
```

One line of ceremony. The JVM guarantees an enum constant exists exactly once — and unlike every version above, it also survives serialization and blocks reflection attacks (yes, reflection can call a private constructor; it cannot instantiate an enum). *Effective Java*: "a single-element enum type is often the best way to implement a singleton."

### The honest caveats (say them out loud)

- A singleton is **global mutable state** wearing a pattern's name. Everything that touches it becomes invisibly coupled to it, and tests can't easily substitute it.
- Notice the tension with everything we've built: our services receive dependencies **through the constructor** — so even if something is conceptually single-instance, prefer *creating one at the composition root and injecting it*. Single instance by **wiring**, not by **class design**. (This is exactly what Spring's default singleton scope does — the pattern got absorbed by DI containers, the same fate as Builder and Strategy.)
- Legitimate remaining uses: truly global, stateless-or-append-only things — loggers, metrics, `Runtime.getRuntime()`.

---

## 2. Functional Interfaces — functions as values

A **functional interface** is an interface with exactly one abstract method. That constraint means a lambda can implement it — the lambda *is* the method body.

### Roll your own

```
java
```

```

@FunctionalInterface                                // compiler enforces "exactly one ab
interface Calculator {
    int calculate(int a, int b);
}

Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;

add.calculate(3, 4);    // 7
mul.calculate(3, 4);    // 12

```

Two *behaviors*, stored in variables, passed around like data. That's the whole idea of functional programming in one snippet.

### The big four (java.util.function) — learn these cold

| Interface                         | Shape                            | Method              | Reads as                 |
|-----------------------------------|----------------------------------|---------------------|--------------------------|
| <code>Function&lt;T, R&gt;</code> | $T \rightarrow R$                | <code>apply</code>  | <i>transform</i>         |
| <code>Predicate&lt;T&gt;</code>   | $T \rightarrow \text{boolean}$   | <code>test</code>   | <i>decide</i>            |
| <code>Consumer&lt;T&gt;</code>    | $T \rightarrow (\text{nothing})$ | <code>accept</code> | <i>do something with</i> |
| <code>Supplier&lt;R&gt;</code>    | $() \rightarrow R$               | <code>get</code>    | <i>produce</i>           |

```

java

Function<String, Integer> length = s -> s.length();
length.apply("Rohit");                // 5

Predicate<Integer> isEven = x -> x % 2 == 0;
isEven.test(10);                      // true
isEven.test(11);                      // false

Consumer<String> print = s -> System.out.println(s);
print.accept("ABC");                  // prints ABC

Supplier<String> idGenerator = () -> UUID.randomUUID().toString();
idGenerator.get();                    // a fresh UUID
idGenerator.get();                    // another one – Supplier PRODU

```

Recognize them by what they're for: `filter` takes a Predicate, `map` takes a Function, `forEach` takes a Consumer, `orElseGet`/lazy defaults take a Supplier.

Useful relatives you'll meet constantly: `BiFunction<T,U,R>` (two inputs), `UnaryOperator<T>` (a `Function<T,T>` — same type in and out, star of §5), `BinaryOperator<T>` (reduce's best friend), `Runnable` `() → nothing`).

## Composition methods — interfaces that combine

These interfaces don't just hold a function; they **combine**:

```
java

Predicate<Integer> isPositive = x -> x > 0;
Predicate<Integer> isEvenAndPositive = isEven.and(isPositive);
Predicate<Integer> isOdd = isEven.negate();

Function<String, String> trim = String::trim;
Function<String, Integer> trimmedLength = trim.andThen(length); // trim FIRST
```

`andThen` is the hinge of this whole class — hold that thought for §5.

## Method references — the shorthand for "this lambda just forwards"

When a lambda does nothing but call an existing method, Java lets you point at the method directly with `::`. You've already been reading them all class — here are the four kinds:

| Kind                                                       | Method reference                 | Equivalent lambda                          |
|------------------------------------------------------------|----------------------------------|--------------------------------------------|
| 1. Static method                                           | <code>Integer::parseInt</code>   | <code>s -&gt; Integer.parseInt(s)</code>   |
| 2. Instance method of a <i>specific</i> object             | <code>System.out::println</code> | <code>s -&gt; System.out.println(s)</code> |
| 3. Instance method of an <i>arbitrary</i> object of a type | <code>String::trim</code>        | <code>s -&gt; s.trim()</code>              |
| 4. Constructor                                             | <code>Question::new</code>       | <code>args -&gt; new Question(args)</code> |

Kind 3 is the one that surprises people: the *first parameter becomes the receiver*. So

`String::compareTo` means `(a, b) -> a.compareTo(b)` — two parameters collapse into "call it on the first, pass the second."

```
java
```

```
Function<String, Integer> length = String::length;           // kind 3: s -> s.len
Supplier<List<String>> maker = ArrayList::new;               // kind 4: () -> new
questions.stream().map(QuestionSummaryResponse::from);      // kind 1: our own D
nums.stream().filter(Main::isEven);                         // kind 1: last clas
```

Two rules of thumb:

- If the lambda **only forwards** (`(x -> foo(x))`), prefer the reference — it removes a layer of noise and names the intent (`(String::trim)` reads better than `(s -> s.trim())`).
- If the lambda does **anything more** — an expression, a condition, two calls — keep the lambda. `(x -> x.trim() + "?")` has no method-reference form, and forcing one by extracting a method is only worth it if the method deserves a name.

Method references are also why good small methods pay double: `(Main::isEven)`, `(QuestionSummaryResponse::from)`, `(Question::score)` — every well-named method you write is automatically a plug-in strategy, for free.

## 3. Streams — a crash course

### 3.1 The shape of a pipeline

| SOURCE                                     | → INTERMEDIATE OPS (0..n)                      | → TERMINAL OP (exactly 1)       |
|--------------------------------------------|------------------------------------------------|---------------------------------|
| where data comes from<br>sum, findFirst... | lazy transformations<br>filter, map, sorted... | the trigger<br>reduce, collect, |

Creating streams:

```
java
Stream.of(a, b, c);           // from values
Arrays.stream(arr);           // from an array
list.stream();                // from a collection
Stream.iterate(1, x -> x + 1); // infinite! (needs a short-circuiting terminal
```

### 3.2 The canonical example

Sum of squares of the odd numbers:

```
java
```

```
int[] arr = {1, 2, 3, 4, 5};
// [1,2,3,4,5] → filter odd → [1,3,5] → square → [1,9,25] → reduce → 35

int sum = Arrays.stream(arr)
    .filter(x -> x % 2 == 1)
    .map(x -> x * x)
    .reduce(0, (a, b) -> a + b);    // or simply .sum()
```

Declarative: the code says **what** (odd → square → sum), not **how** (no index, no accumulator variable, no off-by-one to write wrong).

### 3.3 The big misconception: "three loops"

The arrows above *look* like three passes producing two intermediate lists. **That is not what happens.** Nothing executes until the terminal operation, and then the operations are **fused** into a single pass — each element flows through the *entire* pipeline, one at a time:

java

```
Stream.of(1, 2, 3, 4, 5)
    .filter(x -> { System.out.println("filter " + x); return x % 2 == 1; })
    .map(x -> { System.out.println("map " + x); return x * x; })
    .forEach(x -> System.out.println("SUM ← " + x));
```

```
filter 1      ← 1 goes through the WHOLE pipeline first...
map    1
SUM ← 1
filter 2      ← ...then 2 (rejected – never reaches map)
filter 3
map     3
SUM ← 9
filter 4
filter 5
map     5
SUM ← 25
```

Not (filter, filter, filter, filter, filter, map, map, map, ...) — the interleaving proves it's **one loop, element-at-a-time**, with the three operations fused into its body. Consequences:

- **No intermediate collections** are allocated. [1, 3, 5] and [1, 9, 25] never exist.
- Cost is **O(N) once**, not O(N) per operation. Ten chained ops ≠ ten passes.
- **Laziness enables short-circuiting:** Stream.iterate(1, x -> x + 1).filter(x -> x % 7 == 0).findFirst() terminates instantly on an *infinite* stream — elements are only pulled while someone downstream still wants one.

So chain freely. The readability of small named steps costs you almost nothing — the JIT sees one tight loop.

### 3.4 Parallelism — one word

java

```
int sum = IntStream.rangeClosed(1, 1_000_000).parallel()  
    .filter(x -> x % 2 == 1)  
    .map(x -> x * x)  
    .sum();
```

Because a pipeline declares *what* rather than *how*, the runtime is free to split the source across cores, run the fused pipeline on each chunk, and combine results. Try hand-parallelizing the equivalent `(for)` loop — threads, partitioning, joining, an accumulator race — versus **one word**.

Two honest warnings: (1) parallel has coordination overhead — it pays off for *large* data or *expensive* per-element work, and can be **slower** for small lists; measure. (2) Your lambdas must be **pure** (no shared mutable state), or you've built a race condition with nicer syntax. Which brings us to —

### 3.5 Pure functions

A function is **pure** if its output depends only on its inputs, and it changes nothing outside itself — no I/O, no mutating fields, no clock, no randomness.

java

```
Function<Integer, Integer> square = x -> x * x;           // pure  
Function<Integer, Integer> evil = x -> x + counter++;     // impure: reads A
```

Pure functions are trivially testable, safe to run in parallel, safe to reorder, safe to cache — and, crucially for the next section, **safe to compose**: a chain of pure functions is exactly as predictable as each link.

---

## 4. Decorator Pattern — the old way

**Problem:** add behavior to an object *dynamically*, in any combination, without an inheritance explosion. (Subclassing gives you `BufferedFileInputStream`, `CompressedBufferedFileInputStream`, ... — 2<sup>n</sup> classes. We've seen that number before.)

**Classic solution:** wrap it. A decorator implements the same interface as the thing it decorates, holds a reference to it, and adds its bit around the delegated call. You've been using the most famous example since your first week of Java:

java



```
var in = new DataInputStream(           // adds: readInt(), readDouble()...
    new BufferedInputStream(           // adds: buffering
        new FileInputStream("abc.txt"))); // the actual source
```

Each layer wraps the previous one; each adds one capability; any combination composes. Here's the shape, hand-built:

```
java

interface TextSource {
    String read();
}

class FileText implements TextSource {                // the core
    public String read() { return " how are you"; }
}

class TrimmedText implements TextSource {             // a decorator
    private final TextSource inner;                   // same interface,
    TrimmedText(TextSource inner) { this.inner = inner; } // wraps another o
    public String read() { return inner.read().trim(); } // adds its bit aro
}

class QuestionText implements TextSource {            // another decorator
    private final TextSource inner;
    QuestionText(TextSource inner) { this.inner = inner; }
    public String read() { return inner.read() + "?"; }
}

// compose any combination at runtime:
TextSource source = new QuestionText(new TrimmedText(new FileText()));
source.read();    // "how are you?"
```

It works. But squint at it: **each decorator is a whole class whose entire purpose is to apply one `(String → String)` step**. A class per lambda. We've seen this heaviness before — it's the pre-lambda Strategy ceremony all over again.

## 5. Decorator, modern — function composition

A text transformation is a `(Function<String, String>)`. So let's just write them as functions:

```
java
```

```

Function<String, String> removeTrailingWhitespace = s -> s.trim();
Function<String, String> formatFirstLetter =
    s -> Character.toUpperCase(s.charAt(0)) + s.substring(1);
Function<String, String> addQuestionMark = s -> s + "?";
Function<String, String> capitalize = String::toUpperCase;

```

And "wrapping" becomes `andThen`:

```

java

Function<String, String> transformer =
    removeTrailingWhitespace
        .andThen(formatFirstLetter)
        .andThen(addQuestionMark);

transformer.apply(" how are you");    // "How are you?"

```

That chain is `new QuestionText(new TrimmedText(new FileText()))` — same composition, read left-to-right instead of inside-out, and every decorator went from a class to a line.

### The identity function — composition's "zero"

To fold an *arbitrary list* of decorators into one function, `reduce` needs a starting value that changes nothing. For addition that's `0`; for multiplication, `1`; for composition, it's the function that returns its input untouched:

```

java

Function<String, String> id = Function.identity();    // x -> x

```

`identity().andThen(f)` behaves exactly like `f` — so it's the perfect seed:

### A configurable TextProcessor

```

java

```

```

public class TextProcessor {

    private final Function<String, String> transformer;

    @SafeVarargs
    public TextProcessor(Function<String, String>... decorators) {
        this.transformer = Stream.of(decorators)
            .reduce(Function.identity(), Function::andThen);    // fold: id
    }

    public String processText(String s) {
        return transformer.apply(s);
    }
}

```

java

```

var processor = new TextProcessor(
    removeTrailingWhitespace,
    formatFirstLetter,
    addQuestionMark);

processor.processText("    how are you");    // "How are you?"

// tomorrow's requirement? pass one more function. Zero classes written. OCP by
var shouty = new TextProcessor(removeTrailingWhitespace, capitalize, addQuestionMark);
shouty.processText("    how are you");        // "HOW ARE YOU?"

```

Look closely at that constructor — it's this class's greatest hits in one line: a **Stream** of **functional interfaces**, folded by **reduce** with **identity** as the seed, producing a composed **decorator chain**. Everything today, in one line of code.

**Why pure functions were the prerequisite:** composition is only predictable if each step depends on nothing but its input. One impure decorator (logs, mutates a field, reads a global) and suddenly the *order* and *count* of applications leaks into your program's behavior in ways the types don't show.

**Composition is just math wearing code**

Expressions compose the same way. Take  $f(x) = 3x + x^2$ :

$$\begin{array}{c}
 + \\
 / \quad \backslash \\
 3 \cdot x \quad x^2
 \end{array}
 =
 \begin{array}{c}
 \text{add} \\
 / \quad \backslash \\
 \text{mul}(3, x) \quad \text{square}(x)
 \end{array}$$

```
java
```

```
BinaryOperator<Integer> add = (a, b) -> a + b;  
BinaryOperator<Integer> mul = (a, b) -> a * b;  
UnaryOperator<Integer> square = x -> x * x;  
  
int x = 4;  
int f = add.apply(mul.apply(3, x), square.apply(x));    // 3*4 + 4^2 = 28
```

Small pure functions, snapped together like expressions. That's the functional style in a sentence: **build big behavior by composing small, pure, boring pieces.**

## 6. The running theme: patterns dissolving into functions

Three classes in a row now, the same event has occurred — a GoF pattern collapsed into a language feature:

| Classic pattern    | Its heavyweight form                       | Its modern everyday form                                 |
|--------------------|--------------------------------------------|----------------------------------------------------------|
| Strategy (class 3) | interface + one class per algorithm        | a lambda: <code>sum(nums, x -&gt; x % 2 == 0)</code>     |
| Decorator (today)  | interface + one wrapper class per behavior | <code>f.andThen(g).andThen(h)</code>                     |
| Iterator           | hand-written cursor classes                | <code>for-each</code> , streams                          |
| Command            | interface + one class per action           | <code>Runnable</code> / <code>Consumer</code> in a queue |
| Template Method    | abstract class + subclass per variation    | a method taking a <code>Function</code> parameter        |
| Singleton (today)  | private constructor gymnastics             | an enum — or just DI-container wiring                    |

The GoF wrote for C++ and pre-generics Java. The **problems** they catalogued are permanent; many of the **solutions** were scaffolding for languages that couldn't pass behavior as a value. Modern Java can. So in day-to-day work you'll write `andThen` far more often than you'll write a wrapper class — but when you do, you'll *recognize* it as Decorator, and be able to say so in one word at code review. Vocabulary, not ritual — same conclusion as class 3, now proven twice more.

## 7. Summary

1. **Singleton**: private constructor + one access point. Lazy-with-null-check is a race (check-then-act, like `x++`); the **holder idiom** gets lazy + thread-safe from JVM class-

loading; the **enum** is the bulletproof one-liner. But prefer *one instance by wiring at the composition root* over *one instance by class design* — DI absorbed this pattern.

2. **Functional interfaces:** one abstract method  $\Rightarrow$  a lambda can be the implementation. Know the big four — `Function`/apply, `Predicate`/test, `Consumer`/accept, `Supplier`/get — and their composition methods (`andThen`, `and`, `or`, `negate`).
3. **Method references:** when a lambda only forwards to an existing method, write `::` instead — four kinds (static, specific object, arbitrary object of a type, constructor). `String::compareTo` shows the trick: the first parameter becomes the receiver. Every well-named small method is automatically a plug-in strategy.
4. **Streams:** source  $\rightarrow$  lazy intermediate ops  $\rightarrow$  one terminal op. Operations are **fused into a single element-at-a-time pass** — no intermediate lists, no repeated loops, and laziness enables short-circuiting on infinite streams. Parallelism is one word — but only with pure lambdas, and only worth it for big work.
5. **Pure functions** — output from inputs only, no side effects — are what make composition, parallelism, and testing trivially safe.
6. **Decorator:** classically a wrapper class per behavior (`(new Buffered(new File(...)))`); functionally, `andThen` — with `Function.identity()` as the seed for folding a whole list of decorators into one.